

Memory_Folder Project Documentation

Developer onboarding guide for the local memory, style-learning, and voice-enabled chat system.

Canonical source	docs/PROJECT_DOCUMENTATION.md
Editable report	document/Memory_Folder_Project_Documentation.docx
Shareable export	document/Memory_Folder_Project_Documentation.pdf
Preset	compact_reference_guide

This PDF is generated from the same Markdown source as the DOCX. LibreOffice was not available for DOCX-to-PDF rendering in this environment.

Developer onboarding guide for the local memory, style-learning, and voice-enabled chat system.

Source of truth: this document is grounded in the current repository code, the existing architecture notes, the supplied diagrams in [document/](#), and the dependency manifests captured with the project.

1. Executive Summary

[Memory_Folder](#) is an offline-first personal memory system for a conversational agent. It combines semantic memory retrieval, graph reasoning, adaptive style learning, local or API-backed LLM generation, and optional speech input.

The runtime is centered on [integrated_system.py](#), which wires together:

- [Amem/](#): the memory engine, graph store, vector store, embedding/reranking, DSL, visualization, and local LLM backends.
- [LLM_Style/](#): linguistic feature extraction, emotion analysis, pattern deduplication, and dynamic system-prompt construction.
- [ASR/](#): NeMo-based microphone transcription for voice input.
- [server_llm.py](#): an OpenAI-compatible API client used as a drop-in LLM backend.

The core design pattern is simple: user messages are analyzed, remembered, retrieved, and used to build a per-turn augmented prompt. Memory context is injected only into the current model call so retrieved facts do not leak permanently into chat history.

2. Repository Map

Area	Main files	Purpose
Orchestrator	<code>integrated_system.py</code>	CLI chat loop, runtime arguments, memory factory, style/profile update loop, prompt assembly, commands.
Memory core	<code>Amem/memory.py</code>	<code>OfflineMemory</code> ; add/search/traverse/update/delete/history; coordinates all memory stores.
Retrieval wrappers	<code>Amem/memory_system.py</code>	Normal graph-first search, explicit <code>@remember</code> hybrid search, context construction, command parsing.
Graph store	<code>Amem/graph_store.py</code>	SQLite property graph with nodes, edges, edge properties, similarity/merge utilities, traversal helpers.
Vector store	<code>Amem/vector_store.py</code>	FAISS vector index plus pickle docstore, metadata filters, rebuild/reset support.
Embedding/rerank	<code>Amem/local_embedding.py</code>	Local BGE-M3 embeddings, lexical weights, dense/sparse/ColBERT scoring, graph-enhanced reranking.
Entity index	<code>Amem/entity_index.py</code>	SQLite mapping from entity names to memory IDs, aliases, case-insensitive lookups.
LLM backends	<code>Amem/local_llm.py</code> , <code>server_llm.py</code>	Local GGUF/HF generation and OpenAI-compatible API generation.
Style learning	<code>LLM_Style/linguistic_analyzer.py</code> , <code>LLM_Style/profile_index.py</code> , <code>LLM_Style/pattern_deduplicator.py</code>	Feature extraction, emotion distribution, pattern clustering, style prompt export.
Voice input	<code>ASR/asr_wrapper.py</code>	Lazy-loaded NeMo ASR microphone interface for <code>--voice</code> and <code>@voice</code> .
Config	<code>configs/offline.yaml</code> , <code>Amem/local_config.py</code>	Paths, retrieval weights, reranker settings, model locations, persistence locations.
Data stores	<code>local_data/</code> , <code>system_outputs/</code>	FAISS indexes, SQLite DBs, style profile JSON, graph visualizations, prompt outputs.
Tests	<code>tests/</code> , root <code>test_*.py</code>	Unit/smoke tests for graph, memory, search, and style learning. Some imports are stale; see Known Gaps.

3. System Architecture

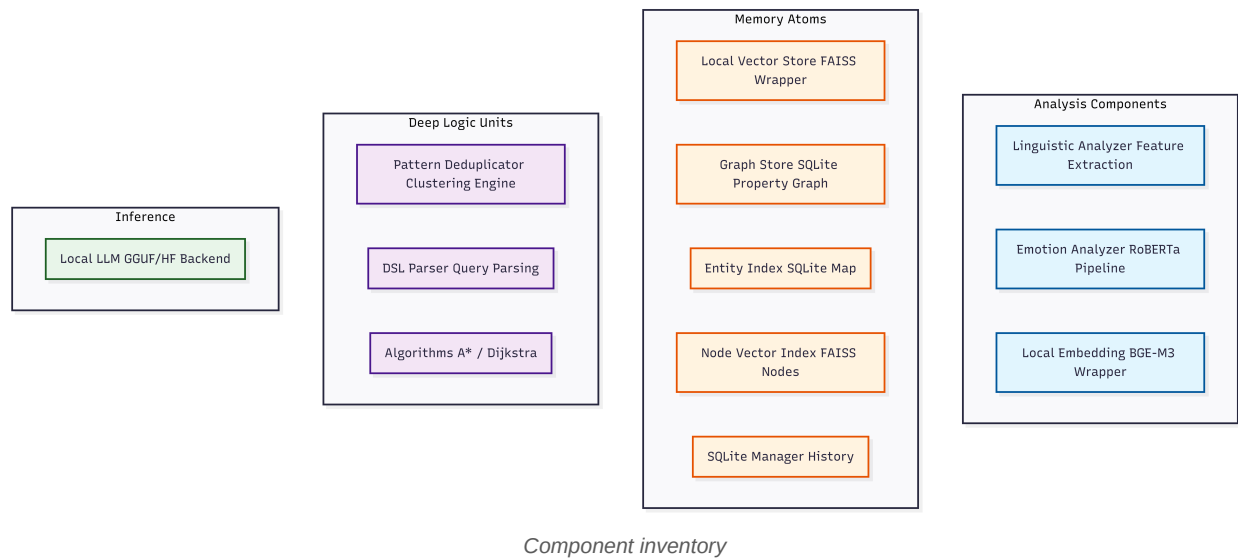


Figure 1. Component inventory. The implementation groups into memory atoms, deep logic units, inference backends, and analysis components.

The system has four major subsystems:

- 1 Memory atoms persist semantic, graph, entity, node-vector, and history state.
- 1 Deep logic units parse DSL queries, deduplicate patterns, and traverse relationship graphs.
- 1 Analysis components extract linguistic features, emotional distributions, and embeddings.
- 1 Inference components generate replies through either local models or OpenAI-compatible server APIs.

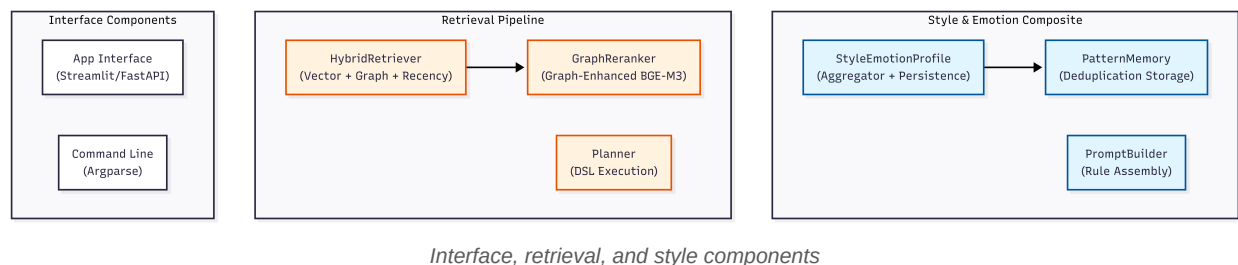


Figure 2. Interface, retrieval, and style composite. The current repo has CLI orchestration and static interface assets; [interface/app.py](#) is referenced by diagrams but is not present as source.

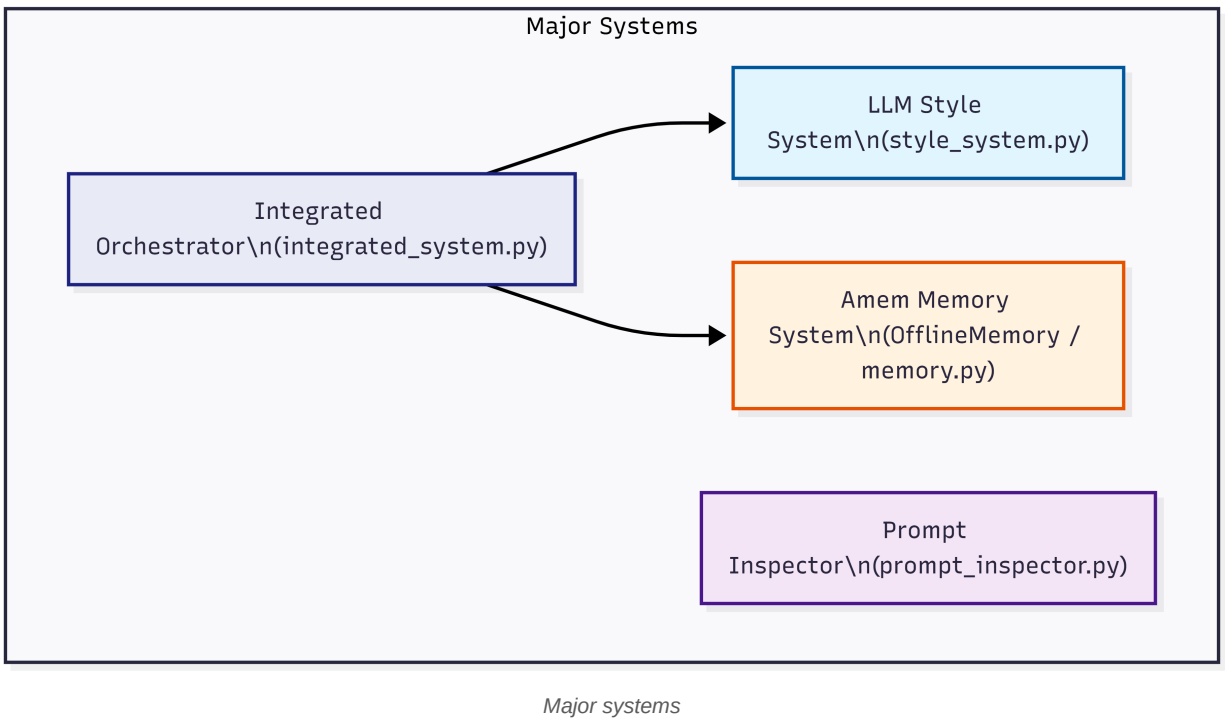


Figure 3. Major systems. `integrated_system.py` currently acts as the executable orchestrator that joins Amem memory, style learning, prompt construction, LLM backend selection, and optional voice input.

4. Runtime Orchestration

`integrated_system.py` is intentionally a wiring layer. Its file header states that business logic lives in `Amem/memory_system.py`, `Amem/memory.py`, and `LLM_Style/`.

Key responsibilities:

- Load `OfflineMemoryConfig` from `configs/offline.yaml`.
- Apply runtime overrides for model paths, generation parameters, index type, reranker top-k, and debug flags.
- Choose an LLM backend:
- `--llm-type local`: use `Amem/local_llm.py`.
- `--llm-type server`: instantiate `ServerLLM`.
- interactive startup asks the user to choose if no type is supplied.
- Initialize `Planner`, `LinguisticAnalyzer`, `EmotionAnalyzer`, and `StyleEmotionProfile`.
- Initialize the strict style pattern deduplicator before style updates.
- Build the base system prompt plus dynamic style/emotion section.
- Maintain chat history while injecting memory context only into the current user message.
- Handle commands such as `@store`, `@remember`, `@delete`, `@path`, `@viz`, `@voice`, and `@eval`.

The chat loop order is:

- 1 Read text or voice input.
- 1 Parse command trigger.
- 1 Detect and persist user/assistant names.

- 1 Update style and emotion profile from the user message.
- 1 Rebuild the dynamic system prompt.
- 1 Retrieve memory context if augmentation is enabled.
- 1 Build a temporary LLM message list with `[Memory Context]` for the current turn.
- 1 Stream model output.
- 1 Store only clean user/assistant messages in history, not the augmented memory block.

5. Configuration and Environment

The default configuration is `configs/offline.yaml`.

Important defaults:

Config area	Default path/value	Notes
LLM models	<code>./models</code>	Local GGUF or HF models. Server mode bypasses local LLM loading.
Embedder	<code>./models/bge-m3</code>	BGE-M3 via <code>FlagEmbedding</code> , 1024 dimensions.
Emotion model	<code>./models/roberta-base-go_emotions</code>	Local RoBERTa GoEmotions model.
Vector store	<code>./local_data/faiss</code>	Stores <code>Amem.faiss</code> and <code>Amem.pkl</code> .
Graph DB	<code>./local_data/graph.db</code>	SQLite property graph.
Entity index	<code>./local_data/entity_index.db</code>	SQLite entity-to-memory mapping.
Node vectors	<code>./local_data/node_vectors</code>	FAISS-backed node/path vector index.
History DB	<code>./local_data/history.db</code>	SQLite audit log for memory changes.
Retrieval	<code>initial_top_k: 50, final_top_k: 10</code>	Search pool and final result sizing.
Reranker	<code>enabled, top_k: 10, max_passage_length: 96</code>	Graph-enhanced BGE-M3 reranking.

The dependency manifests in `document/` show two separate surfaces:

- Core project: `project_pip_packages.txt`, `project_conda_packages.txt`, `requirements.pdf`, root `requirements.txt`.
- ASR: `asr_pip_packages.txt`, `asr_conda_packages.txt`.

For setup, prefer the captured conda manifests because this workspace did not expose a plain `python` command on PATH during inspection. Use the project environment explicitly, for example:

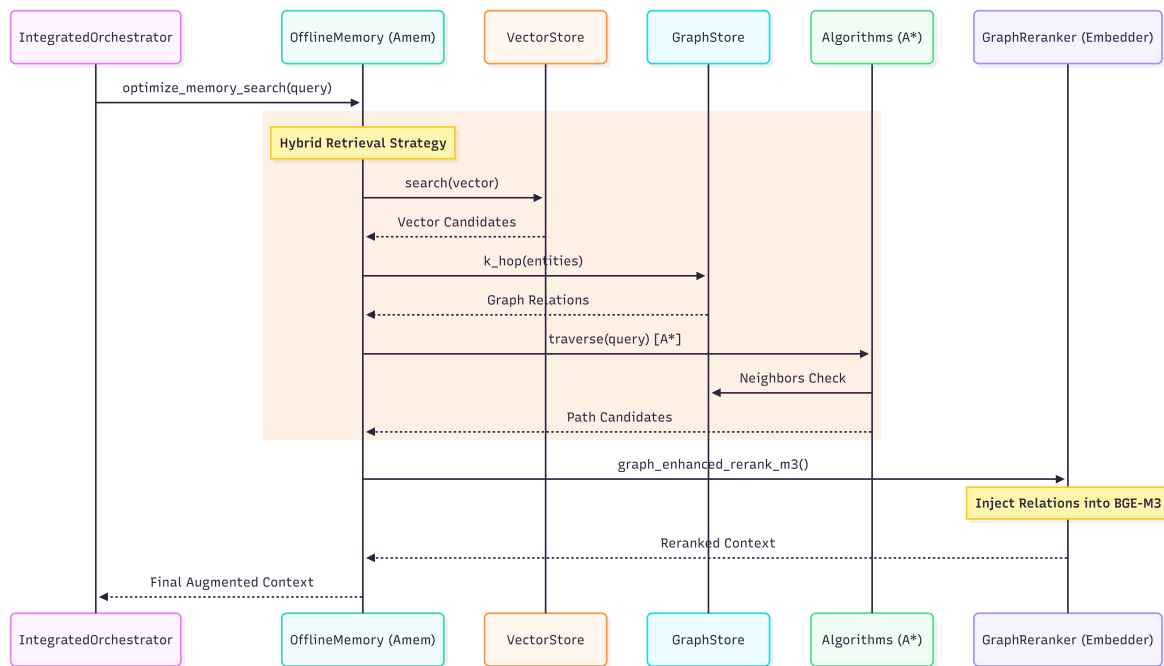
```
conda create --name memory-folder --file document/project_conda_packages.txt
conda activate memory-folder
python integrated_system.py --config configs/offline.yaml
```

For voice features, use the ASR environment manifest or install the ASR stack separately:

```
conda create --name memory-folder-asr --file document/asr_conda_packages.txt
conda activate memory-folder-asr
python integrated_system.py --voice
```

Server mode requires an API key. `server_llm.py` loads `.env` and resolves provider keys from `NVIDIA_API_KEY`, `OPENAI_API_KEY`, or `AZURE_OPENAI_API_KEY` depending on the configured base URL.

6. Memory Engine



Memory class diagram

Figure 4. Memory class relationships. `OfflineMemory` owns the major stores and model wrappers.

`Amem/memory.py` defines `OfflineMemory`, the main in-process memory API. It coordinates:

- `LocalEmbedding` for dense vectors and lexical weights.
- `LocalVectorStore` for semantic recall.
- `GraphStore` for relationship storage and graph traversal.
- `EntityIndex` for entity-to-memory lookup.
- `NodeVectorIndex` for node/path vectors.
- `SQLiteManager` for audit/history storage.
- `LocalLLM` or `ServerLLM` for relationship extraction and final generation.

6.1 Add Pipeline

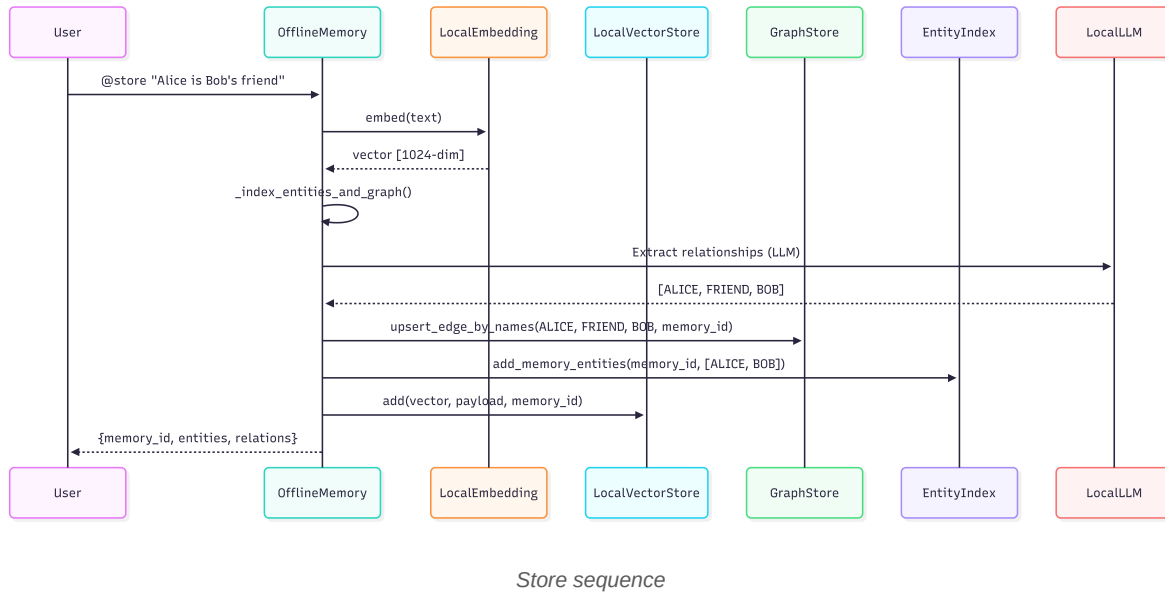


Figure 5. Store sequence. A memory is embedded, analyzed for entities and relationships, inserted into graph/entity stores, added to FAISS, and recorded in history.

`OfflineMemory.add()` accepts either a string or a list of chat messages. It:

- 1 Normalizes the text input and merges `user_id`, `agent_id`, and `run_id` into metadata.
- 1 Embeds the text with BGE-M3 and collects lexical weights.
- 1 Searches for highly similar memories using FAISS.
- 1 If the update threshold is met, merges text into the existing memory and updates the vector payload.
- 1 If not, creates a new UUID memory ID.
- 1 Calls `_index_entities_and_graph()` before writing the vector.
- 1 Stores the vector and payload in `LocalVectorStore`.
- 1 Writes an **ADD** or **UPDATE** record to the history database.

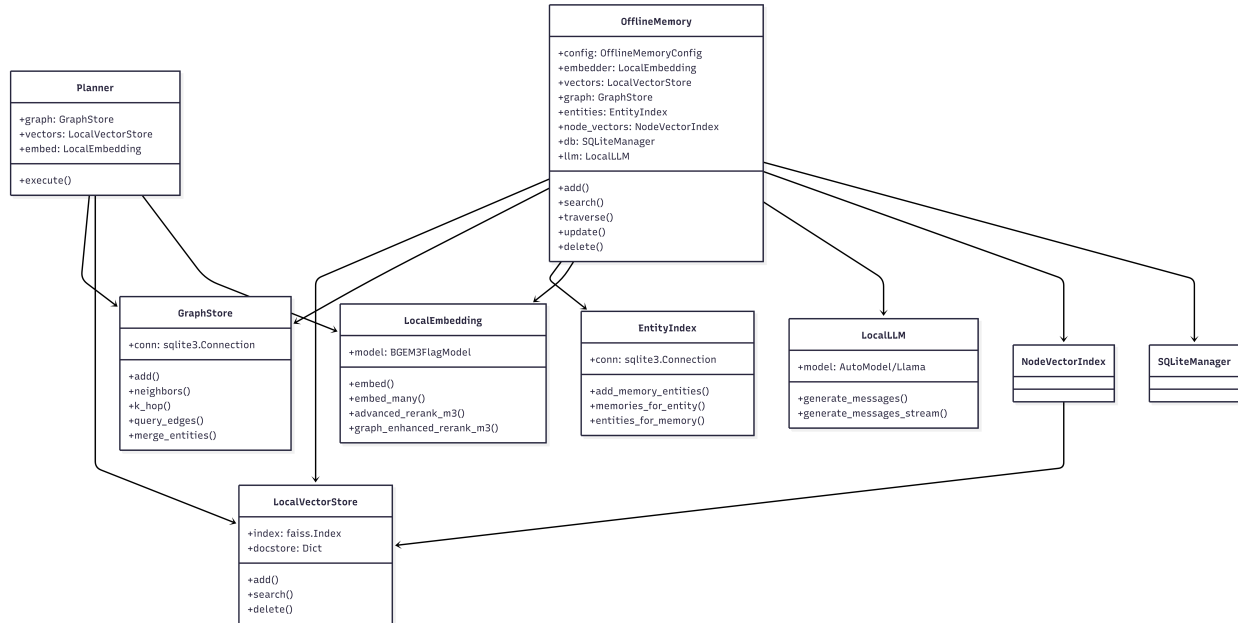
The graph is built before FAISS storage so relationship extraction is attempted even if vector persistence later has problems.

6.2 Entity and Relationship Indexing

The entity pipeline has deterministic and LLM-assisted parts:

- Proper-case and quoted-phrase entities are extracted by `EntityIndex`.
- First-person memories can be associated with the current `user_id`.
- User and assistant names are detected and persisted in `local_data/user_profile.json`.
- Relationship triplets are extracted with graph context and written to `GraphStore`.
- Temporal hints can be attached as edge properties.
- Optional entity and relationship merging exists but is disabled by default in config for performance.

6.3 Search Pipeline



Full hybrid retrieval sequence

Figure 6. Hybrid retrieval sequence. The explicit recall path combines vector candidates, graph relations, A* traversal, and graph-enhanced BGE-M3 reranking.

`OfflineMemory.search()` performs a full hybrid search:

- 1 Embed the query with BGE-M3 and collect lexical query weights.
- 1 Run FAISS vector search with optional metadata filters.
- 1 Detect query entities, including case-insensitive recovery from graph nodes.
- 1 Run A*-style `traverse()` from the `user_id` seed or detected entity.
- 1 Collect memory IDs from traversal nodes and direct query entities.
- 1 Union vector and graph-linked candidates.
- 1 Score candidates using weighted similarity, graph overlap, recency, evidence, and lexical score.
- 1 Build graph context for each top candidate.
- 1 Apply `graph_enhanced_rerank_m3()` if reranking is enabled.
- 1 Return `{"results": [...], "relations": [...]}`.

6.4 Graph Traversal

`OfflineMemory.traverse()` supports possessive-chain style queries such as "Alice's friend's father". It:

- Selects a seed entity, preferring `user_id` when provided.
- Parses possessive terms into a relation chain.
- Runs an A*-style search over graph neighbors.
- Scores edges by relation label match, edge weight, recency, and hub-degree penalty.
- Returns endpoint nodes and path evidence.

For queries without a chain, it ranks direct neighbors.

6.5 Update and Delete

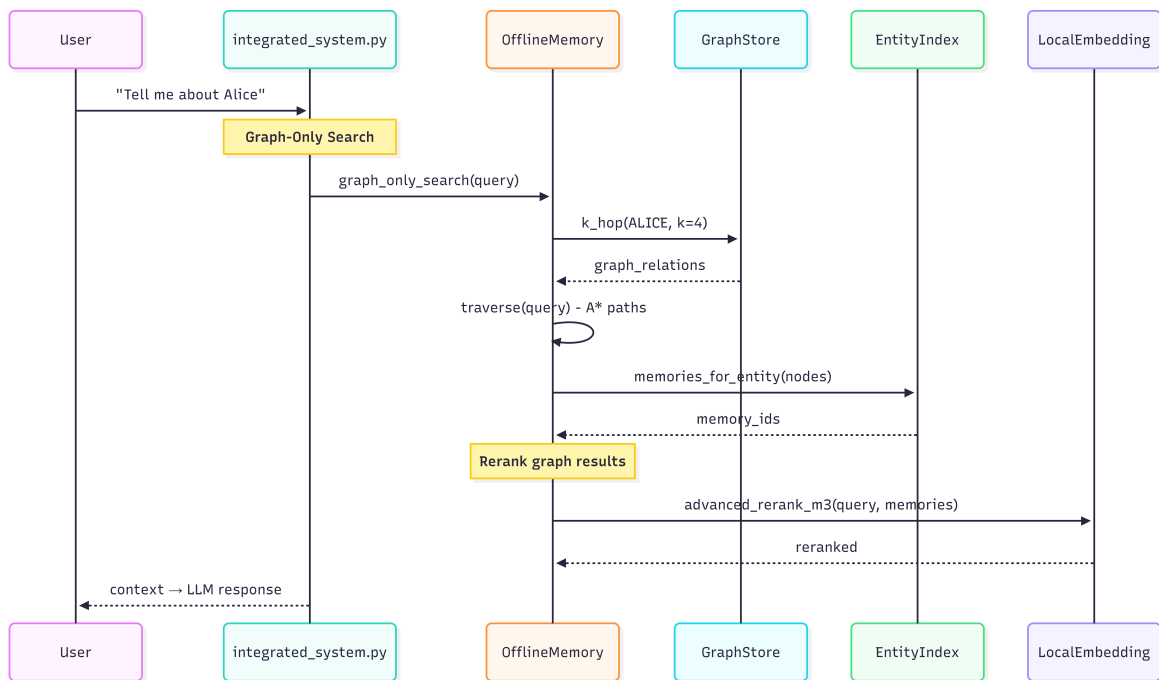
`OfflineMemory.update()` updates vector payloads, removes stale graph edges/path vectors for that memory, re-indexes the new text, and writes history.

`OfflineMemory.delete()` removes a memory from:

- FAISS docstore mapping.
- Graph edges by memory ID.
- Entity index mappings.
- Node/path vectors.
- History log.

`delete_all()` clears all memory stores and resets traversal caches.

7. Retrieval Modes



Remember search sequence

Figure 7. Search sequence for explicit memory recall.

`Amem/memory_system.py` provides two user-facing retrieval modes:

7.1 Normal Chat: Graph-First Search

`graph_only_search()` is used for normal chat turns. It:

- Suppresses memory retrieval for greetings, fillers, and very short messages.
- Extracts entities from the query.
- Runs A* traversal from the user context.
- Collects entity-linked memory IDs.

- Applies BGE-M3 reranking to graph-retrieved memories.
- Filters by relevance threshold.

This prevents random semantic matches from surfacing during lightweight conversation.

7.2 Explicit Recall: @remember

`unified_memory_search()` is used when the user writes `@remember`.

It temporarily disables the internal reranker, increases recall pool size, runs full `mem.search()`, builds context-augmented document strings, reranks with BGE-M3, filters scores, and returns the strongest memories.

7.3 Context Construction

`_get_memory_context()` converts retrieved memories and graph edges into the text block injected into the LLM call:

```
[Memory Context]
Alice --FRIEND--> Bob
Alice is Bob's friend

[User Message]
Who is Alice's friend?
```

The memory block is used only in `messages_for_llm` for the current generation. The permanent chat history stores the original user text, not the augmented context.

8. Graph and Storage Layer

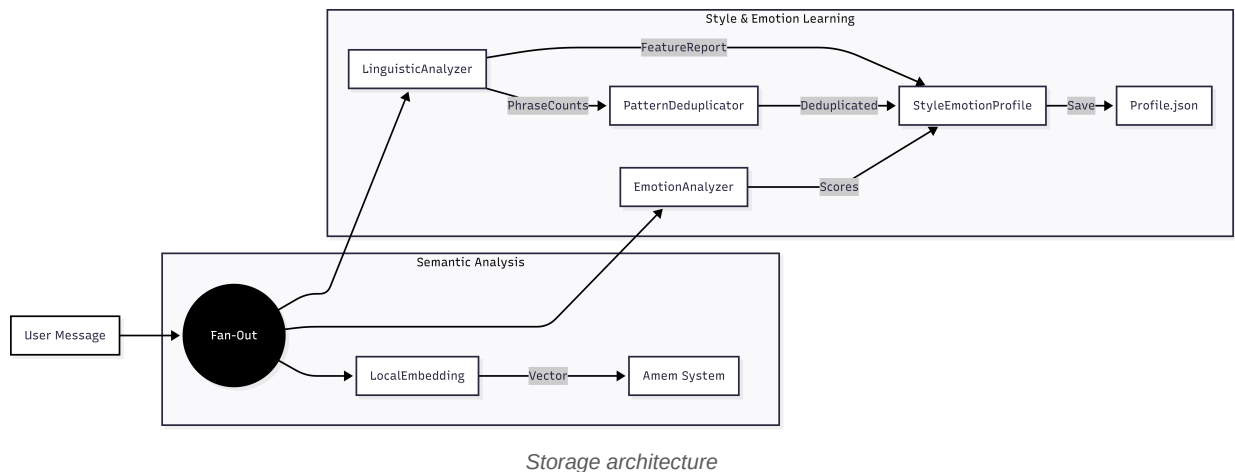


Figure 8. Storage and analysis architecture. The implementation uses SQLite and FAISS local stores.

8.1 GraphStore

`Amem/graph_store.py` implements a SQLite property graph.

Core tables:

- `nodes(id, name, label, props, cluster_id, created_at, updated_at)`
- `node_labels(node_id, label)`
- `node_props(node_id, key, value)`
- `edges(source, relationship, destination, memory_id, source_id, destination_id, created_at, updated_at, weight)`

- `edge_props(source, relationship, destination, key, value)`

Important APIs:

- `upsert()` accumulates edge weight when a triplet already exists.
- `neighbors()` returns in/out/both edges.
- `k_hop()` performs bounded multi-hop expansion.
- `query_edges()` filters relationships.
- `find_similar_entities()` and `find_similar_relationships()` support merge candidates.
- `merge_entities()` and `merge_relationships()` consolidate duplicates.
- `recompute_clusters()` computes connected components.

8.2 LocalVectorStore

`Amem/vector_store.py` wraps FAISS and a pickle docstore. It supports `FLAT`, `HNSW`, and `IVF` index types.

Payloads include:

- `id`
- `memory`
- `hash`
- `metadata`
- stored dense `vector`
- BGE-M3 lexical weights

Deletion removes docstore mappings but does not hard-delete every FAISS entry for all index types. Search skips stale entries through the `idx_to_id` mapping. Heavy churn should be followed by `rebuild_from_docstore()`.

8.3 EntityIndex

`Amem/entity_index.py` stores:

- canonical entities
- aliases
- `memory_id` to entity associations

It is intentionally lightweight and regex-driven. It also supports case-insensitive memory lookup for entity recovery.

8.4 SQLiteManager

`Amem/storage.py` owns history/audit persistence. It records add, update, and delete events and exposes `get_history(memory_id)`.

9. Embeddings and Reranking

`Amem/local_embedding.py` loads BGE-M3 through `FlagEmbedding`. It sets `TRANSFORMERS_OFFLINE=1` by default and resolves local model paths before loading.

Main capabilities:

- `embed(text)`: dense 1024-dimensional embedding.

- `embed_with_lexical(texts)`: dense vectors plus lexical weights.
- `lexical_score()`: BGE-M3 lexical matching score.
- `advanced_rerank_m3()`: query-adaptive dense/sparse/ColBERT fusion.
- `graph_enhanced_rerank_m3()`: appends entity and relationship context to memory text and applies graph-aware boosts.

The advanced fusion logic adapts to query shape:

- factual queries favor sparse/exact matching.
- conceptual queries favor dense semantic matching.
- short queries lean on ColBERT-style fine-grained matching.
- long queries use more dense weighting.

Graph-enhanced reranking adds boosts for entity overlap, graph confidence, and traversal strength.

10. Style and Emotion Learning

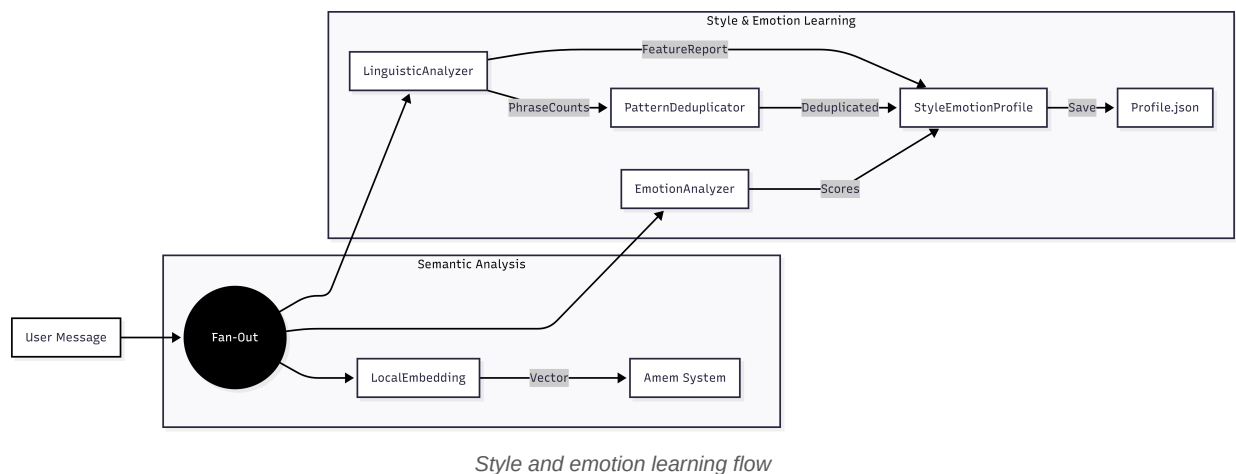


Figure 9. Style and emotion learning. User text fans out to linguistic analysis, emotion analysis, memory storage, and profile persistence.

The style subsystem learns how the user writes and uses that information to rebuild the system prompt.

10.1 LinguisticAnalyzer

`LLM_Style/linguistic_analyzer.py` extracts a `FeatureReport` with:

- structure: sentence length, question/exclamation/period ratios.
- punctuation: commas, dashes, quotes, repeated punctuation, Oxford comma patterns.
- casing: all-caps ratio, lowercase starts, lowercase *i*.
- emoji: counts and positions.
- elongation and repetition.
- lexical categories: fillers, hedges, slang, affective words.
- dynamic user patterns: frequent, rare, and signature words.
- special features: URLs, handles, keysmashes.

- negative style cues: never capitalizing starts, no semicolon/colon, avoids Oxford comma.

It also exports phrase counts used by the deduplication and pattern memory layer.

10.2 StyleEmotionProfile

`LLM_Style/profile_index.py` stores lifetime and session-level style aggregates. It:

- applies EMA decay to recency-sensitive session structures.
- tracks emotion history and average emotion distribution.
- requires explicit `initialize_deduplicator()` before phrase count updates.
- persists deduplication state into `local_data/style_emotion`.
- exports `format_for_llm_system_prompt_detailed()`.

10.3 Pattern Deduplication

`LLM_Style/pattern_deduplicator.py` clusters similar patterns with several strategies:

- direct memory lookup.
- elongation variant clustering.
- filler-specific clustering.
- edit-distance clustering.
- semantic clustering with embedding and FAISS support.

The goal is to avoid treating small variants such as stretched words or spelling variants as entirely separate style cues.

10.4 Emotion Analysis

`EmotionAnalyzer` uses the local `roberta-base-go_emotions` model when available. It outputs the 28 GoEmotions labels and stores normalized distributions in the profile.

The final prompt uses the top emotional signals but should not reveal raw analysis to the user. The documentation intentionally summarizes the prompt-construction process rather than reproducing the full base prompt.

11. Prompt Construction and Inference

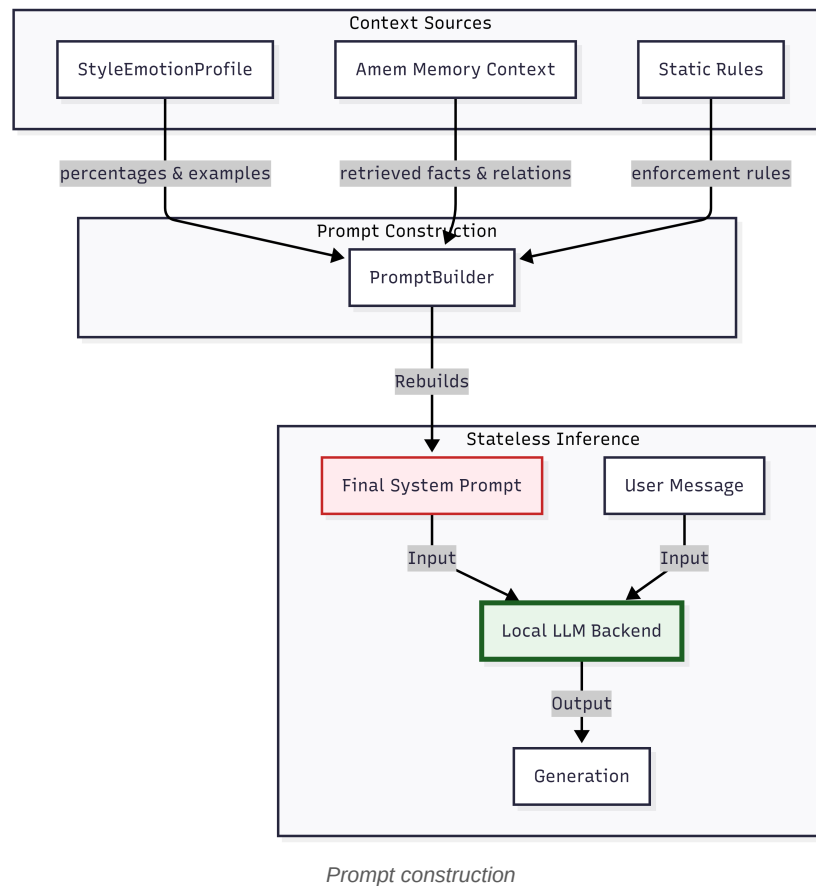


Figure 10. Prompt construction. Style profile, memory context, and static rules feed the final system prompt and current user message.

`compose_system_prompt()` in `integrated_system.py` combines:

- 1 static base behavior rules.
- 1 optional identity block from persisted user/assistant names.
- 1 dynamic style and emotion block from `StyleEmotionProfile`.

Memory context is not part of the permanent system prompt. It is added to the current user message only when retrieval finds relevant context.

12. LLM Backends

12.1 Local LLM

`Amem/local_llm.py` supports:

- GGUF loading through `llama-cpp-python` when `.gguf` files exist and backend is `auto` or `gguf`.
- Hugging Face `transformers` loading otherwise.
- streaming and non-streaming generation.
- Llama 3.1-style message formatting for both GGUF and HF backends.

GGUF loading adapts context, batch size, and thread count based on available memory.

12.2 Server LLM

`server_llm.py` exposes the same `generate_messages()` and `generate_messages_stream()` interface as `LocalLLM`.

It uses the OpenAI SDK against:

- NVIDIA Integrate by default.
- local OpenAI-compatible APIs such as LM Studio or Ollama-compatible servers when a custom base URL is supplied.
- Azure/OpenAI-compatible endpoints through environment variables.

13. Voice Input

`ASR/asr_wrapper.py` is a side-effect-safe wrapper around a NeMo ASR model. It avoids importing `ASR/asr.py` directly because `asr.py` contains its own loop.

Voice flow:

- 1 `@voice` or `--voice` toggles voice mode.
- 1 `ASRInput` lazy-loads `models/nemotron-speech-streaming-en-0.6b/nemotron-speech-streaming-en-0.6b.nemo`.
- 1 A `sounddevice.InputStream` pushes microphone chunks into a queue.
- 1 Energy-based speech/silence detection identifies utterance boundaries.
- 1 NeMo transcribes the buffered audio.
- 1 The resulting text is passed through the same command and chat loop as typed input.

The ASR dependency surface is much heavier than the core project. Keep it isolated unless voice input is required.

14. User Commands

Command	Behavior
<code>@store <text></code>	Stores text as memory, extracts entities/relationships, updates FAISS/graph/entity/history stores.
<code>@remember <query></code>	Runs explicit hybrid vector + graph search and injects retrieved memory context for generation.
<code>@delete all</code>	Deletes all memories from FAISS, graph, entity index, node vectors, and caches.
<code>@delete <query></code>	Deletes matching memory candidates and related entity edges.
<code>@path <entity1> <entity2></code>	Finds graph path using <code>shortest_path()</code> and prints relationship chain.
<code>@viz ALL</code>	Exports the full graph visualization through PyVis.
<code>@viz <entity></code>	Exports a focused graph visualization for matching entities.
<code>@voice</code>	Toggles microphone input.
<code>@eval <jsonl></code>	Runs retrieval evaluation against a JSONL file containing queries and answers.

Normal messages update style/emotion profile and may use graph-first memory augmentation if relevant.

15. Visualization

`Amem/visualize.py` exports PyVis graph views into `local_data/viz/`. The bundled assets under `lib/vis-9.1.2/` and `lib/tom-select/` support local graph viewing without external CDN reliance.

The visualization command gathers FAISS memory texts keyed by graph edge `memory_id` so graph edges can show their underlying memory evidence.

16. Tests and Benchmarks

Test files exist under `tests/` and at the repository root.

Current intended coverage:

- `tests/test_graph.py`: graph weight accumulation and neighbor lookup.
- `tests/test_search.py`: traversal/search smoke behavior.
- `tests/test_memory.py`: add/search/rebuild smoke behavior.
- `tests/test_style_learning.py`: strict style deduplicator initialization, phrase counts, dynamic prompt generation.

Benchmark artifacts include:

- `benchmark_results.json`
- `benchmark_1_results.txt`
- `benchmark_2_results.txt`
- `benchmark_test_data/`

During this documentation work, test collection with the bundled Codex Python runtime could not run because `pytest` was not installed in that runtime. Plain `python` was also not on `PATH`. The project manifests do include `pytest`, so test discovery should be run inside the project conda environment rather than the generic bundled runtime.

Suggested commands inside the project environment:

```
conda activate memory-folder
python -m pytest --collect-only -q
python -m pytest tests -q
```

17. Operations and Troubleshooting

Startup

Use server mode when local LLM weights are unavailable or too heavy:

```
python integrated_system.py --llm-type server --config configs/offline.yaml
```

Use local mode when all local model files are present:

```
python integrated_system.py --llm-type local --config configs/offline.yaml
```

Common Issues

Symptom	Likely cause	Resolution
<code>python</code> not recognized	Environment not activated or Python not on PATH	Activate the conda environment created from the project manifest.
<code>FlagEmbedding</code> not available	Core dependencies missing	Install project packages or use the captured conda environment.
Missing API key in server mode	<code>NVIDIA_API_KEY</code> / <code>OPENAI_API_KEY</code> not set	Add key to <code>.env</code> or shell environment.
Voice input import errors	ASR dependencies not installed	Use the ASR environment manifest.
Random memories in chat	Query is too broad or threshold too low	Normal chat uses graph-first filtering; use <code>@remember</code> for deliberate broad recall.
FAISS stale entries after many deletes	FAISS index cannot hard-delete all entries	Run vector rebuild from docstore in maintenance flow.
Graph visualization empty	No relationship edges present	Store memories with relationship content first, then run <code>@viz</code> .

18. Known Gaps and Current-State Caveats

These are documentation-relevant mismatches found while inspecting the repository:

- Some tests import `mem0.*`, while the current package directory is `Amem/`. These imports should be updated before relying on the tests as-is.
- `tests/test_memory.py` and one path in `Amem/memory_system.py` reference `mem.rebuild_vector_index()`, but the current visible vector rebuild method is `LocalVectorStore.rebuild_from_docstore()`. Confirm or add a wrapper before relying on that path.
- Diagrams and older docs reference `interface/app.py`; the current repo contains `interface/templates/` and `interface/static/`, but no source `interface/app.py`.
- Some existing architecture files and `Amem Memory Architecture.pdf` appear to describe an older line count or method inventory. Treat this document as current to the inspected code.
- Root-level `requirements.txt`, `document/requirements.pdf`, and captured package manifests disagree on a few versions. Prefer the conda manifests that were exported from the working environments.
- The base system prompt in `integrated_system.py` is long and sensitive. Documentation should describe its assembly and behavior, not reproduce it wholesale.

19. Onboarding Checklist

- 1 Create the project environment from `document/project_conda_packages.txt`.
- 1 Confirm model folders exist under `models/`:
 - `bge-m3`
 - `roberta-base-go_emotions`
 - optional local chat model or `.gguf`
 - optional NeMo ASR model
- 1 Configure `.env` for server mode if needed.
- 1 Run `python integrated_system.py --config configs/offline.yaml --llm-type server`.

- 1 Store a simple memory with `@store` Alice is Bob's friend.
- 1 Recall it with `@remember` Who is Alice's friend?.
- 1 Inspect graph output with `@viz` ALL.
- 1 Run tests inside the project conda environment after resolving stale `mem0` imports.

20. Source Material

Primary code files inspected for this documentation:

- `integrated_system.py`
- `server_llm.py`
- `Amem/memory.py`
- `Amem/memory_system.py`
- `Amem/graph_store.py`
- `Amem/vector_store.py`
- `Amem/entity_index.py`
- `Amem/local_embedding.py`
- `Amem/local_llm.py`
- `Amem/local_config.py`
- `Amem/storage.py`
- `Amem/node_vectors.py`
- `LLM_Style/linguistic_analyzer.py`
- `LLM_Style/profile_index.py`
- `LLM_Style/pattern_deduplicator.py`
- `LLM_Style/style_system.py`
- `LLM_Style/prompt_inspector.py`
- `ASR/asr_wrapper.py`

Supporting documentation and artifacts:

- `Architecture.md`
- `Architecture_Diagram.md`
- `docs/architecture.md`
- `docs/api.md`
- `docs/operations.md`
- `docs/PATTERN_DEDUPLICATION_SUMMARY.md`
- `document/Amem Memory Architecture.pdf`
- `document/requirements.pdf`
- `document/project_pip_packages.txt`
- `document/project_conda_packages.txt`

- `document/asr_pip_packages.txt`
- `document/asr_conda_packages.txt`
- `document/Untitled diagram-*.png`